Full Stack Developer Case Study

Project: Mini ERP + CRM Operations Portal (Durga Enterprise)

1. GitHub Repository Link

https://github.com/vinay6314/Durga-Enterprise.git

2. Live Frontend URL

Production Live Frontend: https://durga-enterprise.vercel.app

Local Dev Frontend: http://localhost:3000

3. Live Backend API URL

Production Live Backend API: https://durga-enterprise.onrender.com/api

Health Check Endpoint: https://durga-enterprise.onrender.com/health

Local Dev Backend API: http://localhost:5000/api

4. Test Login Credentials for All Roles

ROLE	EMAIL ADDRESS	PASSWORD	OPERATIONAL PRIVILEGES
ADMIN	vinaychoudary63@gmail.com	**Admin@123**	Full access to CRM, Inventory, Sales, Tax Invoices & Logs
SALES	sales@erp.com	**Sales@123**	Manage Customer CRM Database & Create Sales Challans
WAREHOUSE	warehouse@erp.com	**Warehouse@123**	Product Catalog, Stock IN/OUT Adjustments & Audit Logs
ACCOUNTS	accounts@erp.com	**Accounts@123**	View Sales Challans & Download PDF Tax Invoices

5. Postman Collection & API Documentation

Postman Collection File: ./Postman_Collection.json (Committed in repository root)

Core REST API Endpoints:

- POST /api/auth/login — User authentication & JWT token issuance
- POST /api/auth/register — User registration
- GET /api/auth/me — Current authenticated user profile
- PUT /api/auth/profile — Update display name & password
- GET /api/customers — List & search customers (Paginated)
- POST /api/customers — Add new customer profile
- PUT /api/customers/:id — Edit customer details
- DELETE /api/customers/:id — Delete customer record
- POST /api/customers/:id/follow-up — Add customer follow-up note
- GET /api/products — Product catalog & stock balances
- POST /api/products/stock-movement — Record Stock IN (+) / Stock OUT (-)
- GET /api/products/stock-movements/pdf — Export Stock Audit PDF Report
- POST /api/challans — Create sales challan (auto-deducts stock)
- GET /api/challans/:id/pdf — Download PDF Tax Invoice with Digital Stamp Seal

6. README with Setup and Deployment Instructions

Complete step-by-step documentation is available in README.md & SUBMISSION.md.

Quick Local Run Commands:

Backend: cd backend && npm install && npm run db:setup && npm run dev (Port 5000)

Frontend: cd frontend && npm install && npm run dev (Port 3000)

Docker Containerized: docker-compose up --build

7. Short Explanation of Architecture

Decoupled Tiered REST Architecture:
1. Frontend Layer: Built with React 18, TypeScript, and Vite. Designed with dynamic HSL color tokens supporting Nordic Light & Dark modes, responsive drawers, and SVG stamp seal graphics.
2. Backend API Layer: Express REST API in TypeScript enforcing JWT authentication, authorizeRoles RBAC middleware, and Zod input schema validation.
3. Database Layer: Prisma ORM managing SQLite/PostgreSQL schemas with automatic data seeding and relational cascading deletes.
4. PDF Document Builder: Dynamic vector PDF generation powered by pdfkit for generating print-ready tax invoices and audit log reports.

8. Known Limitations or Incomplete Parts

1. Local SQLite Engine: Uses local SQLite storage (dev.db). For cloud multi-instance auto-scaling deployments, schema.prisma provider can be switched to postgresql.
2. Direct Browser PDF Downloads: PDF downloads trigger direct browser binary downloads (Content-Disposition: attachment).
3. Product Snapshots: Sales Challans store frozen product snapshot data (SKU, product name, price at order creation) to preserve historical invoice integrity.